

BITÁCORA DE USO DE INTELIGENCIA ARTIFICIAL

Fase 4 – Implementación y evidencia

Este documento registra las consultas relevantes realizadas a herramientas de inteligencia artificial durante el diagnóstico, el diseño y la implementación del rediseño. No se evalúa la cantidad de registros, sino la calidad del juicio técnico del equipo: qué se aceptó, qué se corrigió, qué se rechazó y por qué. Cada entrada está alineada con los hallazgos de la Fase 1, el análisis de impacto de la Fase 2 y las decisiones arquitectónicas de la Fase 3.

1. Criterio de decisión del equipo

Antes de aceptar o rechazar cualquier propuesta de la IA, el equipo aplicó los siguientes filtros:

- ¿La propuesta se sostiene con evidencia concreta del código (archivo y línea)?
- ¿Preserva el comportamiento observable del sistema original?
- ¿Resuelve un punto de dolor priorizado o introduce sobre-ingeniería?
- ¿La frontera de responsabilidad propuesta es estable (una sola razón de cambio)?
- ¿Existe una alternativa más simple que cumpla el mismo objetivo sin romper OCP, LSP o ISP?

Aceptado: La propuesta de la IA se incorporó al diseño o al código tal como se formuló (o con ajustes menores de nomenclatura).

Corregido: La dirección de la propuesta era válida, pero el equipo debió ajustar el alcance, la ubicación o la forma de aplicarla.

Rechazado: La propuesta se descartó con argumento técnico. No se aplicó en el TO-BE ni en la implementación.

2. Registro de consultas relevantes

Se presentan las interacciones más significativas. Se omiten consultas puramente mecánicas (syntaxis, nombres de métodos, etc.) y se priorizan aquellas que impactaron decisiones arquitectónicas.

Qué propuso la herramienta	Decisión del equipo	Argumento / justificación técnica
ServicioProducto viola SRP: concentra gestión de productos, persistencia, reglas de inventario y emisión de eventos.	Aceptado	Confirmado por lectura manual del código (H-01). La clase concentra cuatro razones de cambio independientes. Se decidió separar persistencia (IRepositorioProducto), creación (IProductoFactory) y notificaciones (interfaces de notificador), manteniendo la coordinación en ServicioProducto.
ServicioCliente viola SRP: combina gestión, acumulación de puntos, carga desde archivo y eventos.	Aceptado	Coincide con el hallazgo propio H-02. Se separó la persistencia en IRepositorioCliente y la notificación en INotificadorPuntos. La regla de acumulación de puntos se conservó dentro del servicio porque no existe solicitud de cambio autorizada que la convierta en estrategia independiente.
ServicioUsuario viola SRP por administrar usuarios, carga y autenticación.	Corregido	Se acepta la mezcla de responsabilidades, pero se aclara que la autenticación está implementada en AspectoAutenticacion; ServicioUsuario solo la coordina. No se introdujo abstracción sobre

		AspectoAutenticacion porque el equipo determinó que es una regla fija sin variabilidad prevista.
Producto viola SRP por contener MostrarInformacion() con Console.WriteLine.	Aceptado	Hallazgo propio H-03. Se conservó el método como punto de extensión polimórfico para Cosmetico y Comestible, pero se declaró como deuda técnica consciente: el listado del menú (opción 1) no lo invoca y modificarlo cambiaría una salida observable congelada.
Program.cs viola SRP por orquestar demasiadas responsabilidades.	Rechazado	Program actúa como composition root y punto de entrada. Su función de ensamblar dependencias y coordinar la ejecución no constituye por sí misma una violación de SRP. Convertirlo en múltiples clases introduciría complejidad innecesaria sin beneficio real en este contexto.
ServicioProducto viola OCP: CargarDesdeArchivo() crea siempre MedicamentoCapsula.	Aceptado	Hallazgo H-05. Agregar un nuevo tipo obliga a editar la lógica central. Se resolvió desacoplando la lectura (RepositorioProductoArchivo) de la creación concreta (IProductoFactory / ProductoFactory), de modo que incorporar Cosmetico o Comestible solo requiere extender la factory.
Medicamento viola OCP: para soportar un nuevo tipo de medicamento hay que tocar la jerarquía.	Rechazado	Agregar un nuevo tipo de medicamento (ej. MedicamentoEfervescente) no exige modificar Producto, Medicamento ni las subclases existentes: basta con crear una nueva clase que herede de Medicamento. La jerarquía ya es abierta a extensión en ese nivel.
Producto viola OCP: el método virtual MostrarInformacion() está muerto y Program.cs reimplementa la presentación con lógica específica.	Aceptado	Hallazgo H-07. El punto de extensión existe pero no se usa. En el TO-BE se aprovechó para Cosmetico y Comestible (sobrescriben el método), pero el menú principal se dejó sin invocar MostrarInformacion() para no alterar la salida observable. Deuda técnica consciente.
ServicioProducto viola DIP: crea EventoStockMinimo y EventoVencimiento con new en el constructor.	Aceptado	Hallazgo propio H-08. Se introdujeron INotificadorStockMinimo e INotificadorVencimiento, inyectadas por constructor. Los eventos existentes implementan esas interfaces. El composition root (Program) resuelve las instancias concretas.
ServicioCliente y ServicioMovimiento violan DIP por crear sus eventos con new.	Aceptado	Hallazgos H-09 y H-10. Se aplicó el mismo patrón: INotificadorPuntos e INotificadorMovimiento inyectados. Se reutilizó la idea de abstracciones mínimas por evento para preservar ISP y no obligar a un servicio a depender de notificaciones que no usa.
Los tres servicios violan DIP al llamar File.Exists / File.ReadAllLines directamente.	Aceptado	Hallazgos H-11, H-12 y H-13. Se crearon IRepositorioProducto, IRepositorioCliente e IRepositorioUsuario con implementaciones de archivo. Los servicios de alto nivel dejan de conocer el sistema de archivos. Se exigió evaluación perezosa (yield return) para no alterar el comportamiento observable ante fallos de lectura.

ServicioUsuario viola DIP por depender de AspectoAutenticacion estático.	Rechazado	Login() llama a AspectoAutenticacion.Login() sin interfaz. Se descartó porque es una regla de negocio fija sin cambios futuros previstos. Forzar una abstracción no aporta valor y aumentaría la complejidad sin beneficio.
Producto viola DIP por usar Console.WriteLine en MostrarInformacion().	Rechazado	Se clasifica mejor como violación de SRP/OCP (H-03/H-07) que de DIP. La dependencia de un mecanismo de salida concreto se trata en la separación de presentación, no como inversión de dependencia de un módulo de alto nivel.
Crear una clase Repositorio que lea los archivos y, dentro de ella, un switch/case por cada tipo de producto para instanciar la clase concreta.	Rechazado / Corregido	La propuesta inicial de la IA ubicaba el switch de creación dentro del repositorio. Eso perjudica OCP: cada nuevo tipo de producto obligaría a modificar el repositorio. Además, se sugirió 'editar la factory', pero la factory original era estática y no inyectable. Decisión del equipo: el repositorio solo lee líneas y separa campos; la decisión de tipo concreto queda exclusivamente en IProductoFactory (clase de instancia). Así se preserva OCP y se hace inyectable la factory.
Usar una única interfaz IRepositorio genérica con todas las operaciones de lectura (productos, clientes, usuarios).	Rechazado	Violaría ISP. ServicioUsuario terminaría dependiendo de LeerProductos() y LeerClientes(), que nunca usa. Un cambio en la firma de lectura de productos obligaría a revisar el servicio de usuarios sin razón. Se definieron tres interfaces específicas, una por agregado.
Usar una única interfaz INotificador con los cuatro métodos de notificación (stock, vencimiento, puntos, movimiento).	Rechazado	También viola ISP. ServicioMovimiento dependería de NotificarStockMinimo() y NotificarVencimiento(), que no le corresponden. Se definieron cuatro interfaces de una sola operación. IServicioNotificacion (ya existente, un solo método) se conservó sin conectarla al flujo para no alterar salidas observables.
Hacer que Servicio herede de Producto para reutilizar la lógica de venta y movimientos (SC-2).	Rechazado	Violación clara de LSP. Un servicio no tiene stock físico ni fecha de vencimiento reales. Forzarlo a heredar de Producto obligaría a inventar valores sin sentido y generaría alertas falsas. Se decidió separar conceptualmente productos físicos y servicios; en esta fase no se implementó SC-2 para evitar introducir atributos artificiales.
Crear subclases de Cliente por cada tipo de convenio (Empresa, Banco, Universidad, etc.) para SC-3.	Rechazado	Un convenio es una condición comercial asociada al cliente, no un nuevo tipo de cliente. Crear una jerarquía basada en el tipo de convenio produciría herencia inestable (el convenio puede cambiar). Se decidió modelar Convenio por composición/asociación. En esta fase no se implementó SC-3; se dejó como extensión futura sobre la abstracción IDescuento ya existente.
Eliminar por completo MostrarInformacion() de Producto y mover toda la	Rechazado (parcial)	Sería la solución 'pura' de SRP, pero cambiaría el comportamiento observable del listado (opción 1) y rompería el contrato de extensión que ya existe. Se

presentación a una capa de UI separada.		conservó el método como punto polimórfico y se documentó la deuda técnica. Prioridad: no alterar salidas congeladas.
Introducir un contenedor de inyección de dependencias (DI container) para el composition root.	Rechazado	El sistema es una aplicación de consola de tamaño moderado. Program como composition root manual es suficiente, transparente y no introduce dependencias externas. Un contenedor añadiría complejidad y una curva de aprendizaje innecesaria para el alcance del rediseño.
Implementar SC-1, SC-2 y SC-3 en la misma entrega de Fase 4.	Rechazado	El enunciado exige implementar una de las tres. Se eligió SC-1 (cosméticos y comestibles) por ser la más equilibrada: toca poco, no rompe el modelo de dominio existente y confirma que la abstracción Producto ya estaba bien diseñada. SC-2 es la más peligrosa (fuerza herencia incorrecta) y SC-3 es la más volumétrica. Implementar las tres superaría el límite de rediseño y aumentaría el riesgo de alterar comportamiento observable.
Cambiar el formato de productos.txt de inmediato a incluir la columna tipo, sin retrocompatibilidad.	Corregido	Se agregó la séptima columna tipo, pero se mantuvo retrocompatibilidad exacta: una línea de 6 columnas sigue creando MedicamentoCapsula. Así el archivo original de diez productos funciona sin modificación y se preserva el comportamiento observable de la carga inicial.

3. Síntesis del juicio del equipo

De las 22 entradas registradas:

- 11 propuestas fueron aceptadas (principalmente violaciones de SRP, OCP y DIP confirmadas con evidencia propia).
- 3 propuestas fueron corregidas (alcance o ubicación de la solución ajustados por el equipo).
- 8 propuestas fueron rechazadas (sobre-ingeniería, violaciones de ISP/LSP, cambios de comportamiento observable o soluciones que rompían OCP).

El caso más representativo del criterio propio es el del repositorio con switch: la IA propuso una solución que resolvía DIP pero introducía una violación de OCP. El equipo rechazó esa ubicación del switch, movió la decisión de tipo a IProductoFactory (inyectable) y dejó el repositorio solo como lector de líneas. Ese ajuste no apareció en la primera respuesta de la herramienta; surgió del análisis conjunto del equipo.

Asimismo, se rechazó de forma consciente la implementación simultánea de SC-2 y SC-3, y se dejó documentada la deuda técnica de MostrarInformacion() / presentación, priorizando la preservación del comportamiento observable sobre la pureza teórica de SRP.

Esta bitácora demuestra que la IA se usó como apoyo de exploración y confirmación, no como autor de las decisiones. Cada aceptación, corrección o rechazo está respaldado por evidencia del código, por los principios SOLID y por el límite explícito de no alterar la conducta observable del sistema.